Task 1: Time Series Bootstrap and the Ladle Estimator

Zhuoyan Jiang

2026-10-11

Part A: Whitening and Autocovariance Matrices

Overview

This report implements Part A of the assignment: the preprocessing and matrix computations required by the time-series version of the ladle estimator of Nordhausen and Virta (2018). The report has two parts.

- **Section 1** defines the generic functions: centering, whitening, autocovariance estimation, symmetrization, and the AMUSE / SOBI diagonalizers.
- **Section 2** applies these functions to the course data `timeseries.dat` and verifies the implementation numerically.

All computations are implemented directly in R. The only ready-made routines used are `JADE::rjd()` for joint diagonalization (as permitted by the assignment) and `JADE::SOBI()` for an independent verification step. The functions defined here are written so that they can be re-applied inside a bootstrap loop in Part C, where each bootstrap sample must be re-centered, re-whitened, and re-diagonalized.

---

1. Generic functions

1.1 Conventions

The sample size is T (called n in the assignment) and p for the dimension. **Input convention:** every function that takes data expects a numeric $T \times p$ matrix whose rows are the time points, in time order, and whose columns are the variables. This matters in Part C, where every bootstrap sample is again a $T \times p$ matrix: a transposed input would give a wrong result without any error message. The following conventions are used consistently.

- **Centering.** Column-wise subtraction of the sample mean:

$$\bar{x} = \frac{1}{T}\sum_{t=1}^{T} x_t, \qquad \tilde{x}_t = x_t - \bar{x}.$$

- **Scaling.** No additional marginal standardization (each column is not divided by its standard deviation first). Whitening uses the full sample covariance matrix, not only the column standard deviations, so it removes both the differences in scale and the contemporaneous correlation between columns. All scaling is absorbed into the whitening step. The whitening matrix is the symmetric inverse square root

$$\hat{\Sigma}^{-1/2} = V D^{-1/2} V^{\top},$$

  computed from the eigendecomposition $\hat{\Sigma} = V D V^{\top}$ of the sample covariance.

- **Lagging.** For a lag $\tau \geq 1$, the pairs $(x_t, x_{t+\tau})$, with $t = 1, \ldots, T-\tau$, are used. No circular wrap-around in Part A.

- **Normalization.** The $1/T$ normalization is used throughout for the sample covariance and the autocovariance matrices:

$$\hat{\Sigma} = \frac{1}{T}\sum_{t=1}^{T} \tilde{x}_t \tilde{x}_t^\top, \qquad \hat{\Sigma}_\tau = \frac{1}{T}\sum_{t=1}^{T-\tau} \hat{x}_t^{st} \hat{x}_{t+\tau}^{st\top}.$$

  Here $\hat{x}_t^{st}$ denotes the *whitened* series, so the autocovariance is estimated on the whitened data. The $1/T$ normalization is the same convention as R's `acf(..., type = "covariance", demean = FALSE)`.

- **Symmetrization.** Under the model, Σ_τ is symmetric. The finite-sample estimate $\hat{\Sigma}_\tau$ is in general not symmetric, and its antisymmetric part is purely sampling noise. That part is removed:

$$\tilde{\Sigma}_\tau = \frac{1}{2}\left(\hat{\Sigma}_\tau + \hat{\Sigma}_\tau^\top\right).$$

  Under the model, symmetrization removes sampling noise and does not remove signal.

- **Definition of Λ and ordering.** Let $\tilde{\Sigma}_\tau$ be the symmetrized autocovariance matrix of the whitened series. The vector `Lambda` returned by these functions contains the diagonal elements of the matrix Λ in the paper, that is, the values λ_j^2:

  - AMUSE (one lag τ): $\lambda_j^2 = \left(u_j^\top \tilde{\Sigma}_\tau u_j\right)^2$.
  - SOBI (lag set $\mathcal{T}$): $\lambda_j^2 = \sum_{\tau\in\mathcal{T}} \left(v_j^\top \tilde{\Sigma}_\tau v_j\right)^2$.

  Here u_j and v_j are the columns of the estimated orthogonal matrix that belong to λ_j^2. In both cases the columns are ordered so that $\lambda_0^2 \geq \lambda_1^2 \geq \cdots \geq \lambda_{p-1}^2$. The paper indexes these values from 0 to $p-1$, so `Lambda[k + 1]` in R is λ_k^2. Part C uses the same sorted `Lambda` and the same column order. In the theory the orthogonal matrix is called U. In the code it is called `U` for AMUSE and `V` for SOBI (`V` is the name used by `rjd()`).

1.2 Why whitening

In the second-order source separation (SOS) model

$$x_t = \mu + \Omega z_t, \qquad \mathrm{Cov}(z_t) = I_p,$$

the covariance of x_t is $\Sigma = \Omega\Omega^\top$, so the matrix $\Sigma^{-1/2}\Omega$ is orthogonal. The whitened series is therefore

$$x_t^{st} = \Sigma^{-1/2}(x_t - \mu) = U z_t, \qquad U^\top U = I_p.$$

This has three consequences.

1. The unknown mixing matrix Ω reduces to an **orthogonal** matrix U.
2. The lag-τ autocovariance becomes

$$\Sigma_\tau = U\, D_\tau\, U^\top, \qquad D_\tau = \mathrm{Cov}(z_t, z_{t+\tau}),$$

   If the component processes are mutually independent (assumptions (A1) and (A2) of Nordhausen and Virta, 2018), or at least uncorrelated across components at every lag, then D_τ is diagonal. The condition $\mathrm{Cov}(z_t) = I_p$ alone is not enough, because it only describes the components at the same time point. The diagonal form of D_τ is the condition that lets all the whitened lagged autocovariance matrices share the same diagonalizing matrix U. So a single lag can be diagonalized by `eigen()` (AMUSE), and a set of lags can be jointly diagonalized by an orthogonal matrix (SOBI).
3. Whitening removes scale and contemporaneous correlation; what is left in $\tilde{\Sigma}_\tau$ is the serial dependence that distinguishes signal components from white noise.

1.3 Functions

```r
center_series <- function(X) {
  # Subtract the column means
  mu_hat <- colMeans(X)
  Xc <- sweep(X, 2, mu_hat, FUN = "-")
  list(mu = mu_hat, Xc = Xc)
}

whiten_series <- function(Xc) {
  Tn <- nrow(Xc)
  p <- ncol(Xc)

  # Sample covariance (1/T) and its eigendecomposition, which must not be singular
  S <- crossprod(Xc) / Tn
  eig <- eigen(S, symmetric = TRUE)
  stopifnot(min(eig$values) > 1e-10 * max(eig$values))

  # Symmetric inverse square root and whitened data
  D_inv_sqrt <- diag(1 / sqrt(eig$values), nrow = p)
  S_inv_sq <- eig$vectors %*% D_inv_sqrt %*% t(eig$vectors)

  Xw <- Xc %*% S_inv_sq

  list(S = S, eig = eig, S_inv_sq = S_inv_sq, Xw = Xw)
}

# Lag-tau autocovariance of the whitened series: (1/T) * sum_t x_t x_{t+tau}'
autocov_raw <- function(Xw, tau) {
  # The lag must be valid
  stopifnot(tau >= 1, tau < nrow(Xw))
  Tn <- nrow(Xw)
  # Pairs (x_t, x_{t+tau}) for t = 1, ..., T - tau
  A <- Xw[1:(Tn - tau), , drop = FALSE]
  B <- Xw[(1 + tau):Tn, , drop = FALSE]
  crossprod(A, B) / Tn
}

# Remove the antisymmetric part
symmetrize <- function(M) {
  0.5 * (M + t(M))
}

# AMUSE: Lambda_j = (u_j' Sigma_tau u_j)^2 for ONE lag, sorted decreasingly
amuse_fit <- function(Sigma_tau_sym) {
  # The eigenvalues are u_j' Sigma_tau u_j
  eig <- eigen(Sigma_tau_sym, symmetric = TRUE)
  # Sort by lambda^2, not by the signed eigenvalue
  ord <- order(eig$values^2, decreasing = TRUE)
  list(
    U = eig$vectors[, ord, drop = FALSE],
    diag_vals = eig$values[ord],
    Lambda = eig$values[ord]^2
  )
}
```

```r
sobi_fit <- function(X, lags = 1:12) {
  # Center, whiten and compute the symmetrized autocovariances at all lags
  cen <- center_series(X)
  wht <- whiten_series(cen$Xc)
  Xw <- wht$Xw
  p <- ncol(Xw)

  S <- array(NA_real_, dim = c(p, p, length(lags)))
  for (k in seq_along(lags)) {
    S[, , k] <- symmetrize(autocov_raw(Xw, lags[k]))
  }

  # Joint diagonalization
  V <- rjd(S)$V

  # SOBI: Lambda_j = sum over lags of (v_j' Sigma_tau v_j)^2, then sorted decreasingly
  Lambda_raw <- Reduce(`+`, lapply(seq_along(lags), function(k) {
    diag(t(V) %*% S[, , k] %*% V)^2
  }))
  ord <- order(Lambda_raw, decreasing = TRUE)

  list(V = V[, ord, drop = FALSE], Lambda = Lambda_raw[ord], Xw = Xw, S = S,
       W = wht$S_inv_sq, mu = cen$mu)
}
```

---

2. Application to `timeseries.dat`

2.1 Data

```r
ts_data <- as.matrix(read.table("timeseries.dat", header = FALSE))

cat("Dimension of the data (T x p):", dim(ts_data))
```

```
## Dimension of the data (T x p): 1000 10
```

```r
# Any missing values?
anyNA(ts_data)
```

```
## [1] FALSE
```

The data have $T = 1000$ time points and $p = 10$ variables, so the input convention of Section 1.1 holds (rows are time points and $T > p$). There are no missing values.

2.2 Centering and whitening

```r
# Center and whiten the data
cen <- center_series(ts_data)
mu_hat <- cen$mu
Xc <- cen$Xc

wht <- whiten_series(Xc)
S <- wht$S
Xw <- wht$Xw
```

```r
cat("Column means after centering (should be approximately zero):")
```

```
## Column means after centering (should be approximately zero):
```

```r
print(colMeans(Xc))
```

```
##            V1            V2            V3            V4            V5
## -2.459144e-17  3.352874e-17  5.996981e-15  5.506706e-17 -1.078998e-17
##            V6            V7            V8            V9           V10
##  2.358114e-16 -3.828049e-16  4.718448e-18  3.632650e-16 -5.206946e-17
```

```r
cat("Eigenvalues of the sample covariance matrix:")
```

```
## Eigenvalues of the sample covariance matrix:
```

```r
print(wht$eig$values)
```

```
##  [1] 4.313670 3.783830 3.316384 2.710873 2.551159 2.333506 1.822926 1.448601
##  [9] 1.250509 1.003756
```

The column means after centering are zero up to rounding error. All eigenvalues of the sample covariance matrix are positive (the smallest is about 1.0), so the inverse square root $\hat{\Sigma}^{-1/2}$ exists.

2.3 Numerical verification of whitening

The whitened sample covariance should be close to I_p:

$$\frac{1}{T}\sum_{t=1}^{T} \hat{x}_t^{st}\hat{x}_t^{st\top} \approx I_p.$$

Because we use the $1/T$ normalization, the check is against I_p. Had we used R's `cov()` (which divides by $T-1$), the result would be $\frac{T-1}{T} I_p \approx 0.999 I_p$, not I_p.

```r
# The whitened covariance should be the identity
S_white <- crossprod(Xw) / nrow(Xw)
err_F <- sqrt(sum((S_white - diag(ncol(Xw)))^2))
off_max <- max(abs(S_white - diag(ncol(Xw))))

cat("Frobenius distance ||S_white - I||_F =", signif(err_F, 4))
```

```
## Frobenius distance ||S_white - I||_F = 9.655e-15
```

```r
cat("Max absolute deviation from I =", signif(off_max, 4))
```

```
## Max absolute deviation from I = 2.998e-15
```

Both quantities are at machine precision, confirming the whitening has the intended effect on the sample covariance matrix.

2.4 Autocovariance matrices

The autocovariance matrices of the whitened series are estimated at lags 1 to 12 (the SOBI lag set used in Nordhausen and Virta, 2018) and symmetrize them as described in Section 1.1. The matrix at lag 1 is used by AMUSE. `sobi_fit()` recomputes all twelve matrices from the raw data, so that it can be applied to bootstrap samples in Part C.

```r
# Symmetrized autocovariance matrices at lags 1 to 12
lags <- 1:12
```

```r
p <- ncol(Xw)

Sigma_sym_list <- lapply(lags, function(tau) {
  symmetrize(autocov_raw(Xw, tau))
})
```

2.5 Independent check of the autocovariances

The matrices are compared with R's multivariate `acf(..., type = "covariance", demean = FALSE)`, which computes the same $1/T$ sums of lagged products, including the cross terms between different columns. Only the symmetrized matrices are used later, so both sides are symmetrized before the comparison. This also means that the check does not depend on the order of the two indices in the array returned by `acf()`.

```r
# Compare with acf(); both sides are symmetrized
a <- acf(Xw, lag.max = 12, type = "covariance",
         plot = FALSE, demean = FALSE)$acf

diffs <- sapply(lags, function(k) {
  max(abs(Sigma_sym_list[[k]] - symmetrize(a[k + 1, , ])))
})

cat("Max absolute difference versus acf() over all lags and entries:",
    signif(max(diffs), 4))
```

```
## Max absolute difference versus acf() over all lags and entries: 0
```

The difference is zero up to rounding error, so the autocovariance matrices agree with R's own computation.

2.6 AMUSE (two-matrix / eigenvalue case)

AMUSE uses a single lag τ and finds an orthogonal matrix U_τ that diagonalizes $\tilde{\Sigma}_\tau$:

$$U_\tau^\top \tilde{\Sigma}_\tau U_\tau = \text{diagonal}, \qquad \Lambda = \operatorname{diag}\left(U_\tau^\top \tilde{\Sigma}_\tau U_\tau\right)^2.$$

The columns of U_τ are reordered so that the corresponding Λ values are decreasing. `amuse_res$Lambda` contains $(\lambda_0^2, \ldots, \lambda_{p-1}^2)$ as defined in Section 1.1. The lag is $\tau = 1$.

```r
# AMUSE at lag 1
tau_amuse <- 1
amuse_res <- amuse_fit(Sigma_sym_list[[which(lags == tau_amuse)]])

cat("AMUSE at lag tau =", tau_amuse)
```

```
## AMUSE at lag tau = 1
```

```r
cat("Lambda (decreasing):")
```

```
## Lambda (decreasing):
```

```r
print(round(amuse_res$Lambda, 4))
```

```
##  [1] 0.0558 0.0423 0.0146 0.0084 0.0052 0.0025 0.0019 0.0016 0.0011 0.0000
```

2.7 SOBI (joint diagonalization via `JADE::rjd()`)

SOBI uses the lag set $\mathcal{T} = \{1, \ldots, 12\}$ and jointly diagonalizes $\tilde{\Sigma}_{\tau_1}, \ldots, \tilde{\Sigma}_{\tau_K}$ by an orthogonal matrix $U_\mathcal{T}$:

$$\Lambda = \sum_{\tau \in \mathcal{T}} \operatorname{diag}\left(U_{\mathcal{T}}^{\top} \tilde{\Sigma}_{\tau} U_{\mathcal{T}}\right)^2 .$$

We use the function `sobi_fit()` defined in Section 1.3, which will also be used in Part C for each bootstrap sample.

```
# SOBI with lags 1 to 12
sob <- sobi_fit(ts_data, lags = 1:12)
V <- sob$V

cat("SOBI Lambda (decreasing):")
```

```
## SOBI Lambda (decreasing):
```

```
print(round(sob$Lambda, 4))
```

```
##  [1] 0.3986 0.3419 0.1397 0.0335 0.0310 0.0247 0.0246 0.0198 0.0173 0.0125
```

```
cat("Orthogonality error ||V'V - I||_max =",
    signif(max(abs(crossprod(V) - diag(ncol(V)))), 4))
```

```
## Orthogonality error ||V'V - I||_max = 2.22e-15
```

`sob$Lambda` contains $(\lambda_0^2, \ldots, \lambda_{p-1}^2)$ as defined in Section 1.1, so Part C can use it directly. The orthogonality error is at machine precision, so V is orthogonal.

As an independent check, we compare our components with those of the ready-made `JADE::SOBI()`. This function is used only for this check. Because the sign and the order of components are not identifiable, we compare them by absolute correlation. For each of our components we report the best match and the second-best match among the `JADE::SOBI()` components.

```
# Compare with JADE::SOBI, up to sign and order
ref <- JADE::SOBI(ts_data, k = 12)
C <- abs(cor(sob$Xw %*% V, ref$S))

best <- apply(C, 1, max)
second <- apply(C, 1, function(r) sort(r, decreasing = TRUE)[2])

cat("Smallest best-match correlation:", signif(min(best), 4))
```

```
## Smallest best-match correlation: 0.9996
```

```
cat("Largest second-best correlation:", signif(max(second), 4))
```

```
## Largest second-best correlation: 0.02745
```

Each of our components has one match with absolute correlation close to 1, and all other correlations are close to 0. Our solution and `JADE::SOBI()` therefore produce the same components up to sign and order.

---

3. Summary of Part A

- **Conventions.** Column centering; no additional scaling; positive lags only; $1/T$ normalization (same as R's `acf(type = "covariance", demean = FALSE)`); symmetrization of the sample autocovariance matrices.
- **Whitening.** The series is whitened by the symmetric inverse square root $\hat{\Sigma}^{-1/2}$. Numerical verification shows the whitened sample covariance is at machine precision equal to I_p.

- **Autocovariances.** Estimated at lags 1 to 12 with $1/T$ normalization and no wrap-around. The antisymmetric part of the raw estimates is sampling noise and is removed by symmetrization. The symmetrized estimates agree with `acf()` to machine precision.
- **AMUSE.** One lag, base R `eigen()`, giving $\Lambda = \mathrm{diag}(U_\tau^\top \tilde{\Sigma}_\tau U_\tau)^2$, ordered decreasingly.
- **SOBI.** Lags 1 to 12, joint diagonalization via `JADE::rjd()`, giving $\Lambda = \sum_\tau \mathrm{diag}(U_\mathcal{T}^\top \tilde{\Sigma}_\tau U_\mathcal{T})^2$, ordered decreasingly. Orthogonality of V is at machine precision.
- **Verification.** An independent check with `JADE::SOBI()` confirms our SOBI solution agrees with the reference implementation up to sign and order.

The functions `center_series()`, `whiten_series()`, `autocov_raw()`, `symmetrize()`, `amuse_fit()`, and `sobi_fit()` defined in Section 1.3 will be reused in Part C, where each bootstrap sample must be re-centered, re-whitened, and re-diagonalized. Part B only requires the original data matrix `ts_data`.

Part B: Bootstrap for Dependent Observations

Overview

Part B implements two bootstrap methods for dependent multivariate observations: the fixed-block bootstrap and the stationary bootstrap (SB) of Politis and Romano (1994).

Section 1 gives the methods and the implementation. Section 2 shows how the methods behave on a strongly dependent simulated series. Section 3 checks that the same functions work on the course data as a general $T \times p$ matrix. Section 4 summarises.

Notation: T is the number of time points, p the dimension, and X the $T \times p$ data matrix with rows as time. The SB parameter is called p_{sb} to avoid a clash with the dimension p.

---

1. Bootstrap functions

1.1 Why iid resampling is not appropriate

The iid bootstrap draws $X_1^*, \ldots, X_T^*$ independently from $\{X_1, \ldots, X_T\}$ with replacement. The draws are independent, so the resampled series has no serial dependence: $\mathrm{Cov}^*(X_t^*, X_{t+\tau}^*) = 0$ for every $\tau \neq 0$, whatever the dependence in the original series. For the ladle estimator this is a problem, since it is built from the autocovariance matrices Σ_τ. An iid bootstrap sample has autocovariance matrices that only reflect noise around zero, so the eigenvectors and the f part of the ladle would be computed from the wrong distribution.

Block resampling instead keeps consecutive observations together: the dependence inside each block is preserved, and only the joins between blocks break it. Since whole rows X_t are resampled (Politis and Romano, 1994, Section 4.1), the dependence between the p components at the same time point, and between components at nearby lags, is also kept.

1.2 Fixed-block bootstrap

Let B be the block length and $h = \lceil T/B \rceil$. A bootstrap sample is generated as follows (Nordhausen and Virta, 2018, Section 2).

1. Draw h start points $s_1, \ldots, s_h$ uniformly from $\{1, \ldots, T - B + 1\}$.
2. Block j is $(X_{s_j}, \ldots, X_{s_j+B-1})$.
3. The blocks are concatenated and the series is truncated to length T.

The tuning parameter is the block length B. Blocks do not wrap around, so the first and last observations are covered by fewer blocks than the middle ones, and the resampled series is not stationary. With $B = 1$ the method reduces to the iid bootstrap.

1.3 Stationary bootstrap

The SB uses blocks with random lengths. Let $p_{\text{sb}} \in (0, 1]$ be the tuning parameter.

1. Draw a start point s uniformly from $\{1, \dots, T\}$ and a block length L with

$$P(L = m) = (1 - p_{\text{sb}})^{m-1} p_{\text{sb}}, \qquad m = 1, 2, \dots$$

   This is the geometric distribution on $\{1, 2, \dots\}$; in R this is `rgeom(1, p_sb) + 1`.
2. The block is $(X_s, \dots, X_{s+L-1})$ with indices taken circularly: $X_{T+k} = X_k$.
3. Blocks are added until the series has length T; the last block is cut.

Since $E(L) = 1/p_{\text{sb}}$, the *expected block length* is $\ell = 1/p_{\text{sb}}$, and a target ℓ is obtained with $p_{\text{sb}} = 1/\ell$. The tuning parameter is p_{sb}, or equivalently the expected length ℓ. With $p_{\text{sb}} = 1$ all blocks have length one and the method reduces to the iid bootstrap. The circular wrapping combined with the memoryless geometric lengths makes the SB pseudo-series stationary conditional on the data (Politis and Romano, 1994, Proposition 1).

1.4 Differences between the two methods

The SB uses random geometric block lengths with mean $1/p_{\text{sb}}$ and wraps the data around a circle, whereas the fixed-block bootstrap uses a constant block length and does not wrap. As a consequence the SB pseudo-series is conditionally stationary while the fixed-block pseudo-series is not. In the SB, τ consecutive steps stay inside the same block with probability $(1 - p_{\text{sb}})^{\tau}$, so the dependence at lag τ decays geometrically. In the fixed-block bootstrap, dependence is kept only for lags shorter than B.

1.5 Implementation

All functions take a numeric $T \times p$ matrix with rows as time and resample whole rows with a common index, so the dependence between components is kept.

```
# Index generators: each returns a vector of T row indices.
iid_index <- function(T) sample.int(T, T, replace = TRUE)

fixed_block_index <- function(T, B) {
  # h blocks of length B from random start points, cut to length T
  h <- ceiling(T / B)
  starts <- sample.int(T - B + 1, h, replace = TRUE)
  idx <- rep(starts, each = B) + rep(0:(B - 1), times = h)
  idx[1:T]
}

stationary_index <- function(T, p_sb) {
  # Geometric block lengths, enough to cover T
  lens <- integer(0)
  while (sum(lens) < T) {
    lens <- c(lens, rgeom(ceiling(2 * T * p_sb) + 10, p_sb) + 1)
  }
  # Random start points, circular indices
  lens <- lens[1:which(cumsum(lens) >= T)[1]]
  starts <- sample.int(T, length(lens), replace = TRUE)
  idx <- rep(starts, lens) + sequence(lens) - 1
  ((idx - 1) %% T + 1)[1:T]
}

# method is "iid", "fixed" (tuning = B) or "stationary" (tuning = p_sb).
boot_index <- function(T, method, tuning = NULL) {
```

```r
  switch(method,
         iid = iid_index(T),
         fixed = fixed_block_index(T, tuning),
         stationary = stationary_index(T, tuning))
}

# List of M bootstrap samples, each a T x p matrix.
boot_samples <- function(X, M, method, tuning = NULL) {
  X <- as.matrix(X)
  lapply(seq_len(M), function(m)
    X[boot_index(nrow(X), method, tuning), , drop = FALSE])
}
```

`boot_samples()` returns M matrices of size $T \times p$, which can be passed to the SOBI function of Part A to compute the ladle quantities.

---

2. Application to simulated data

The two methods are applied to a bivariate VAR(1) series with strong serial dependence, so that the difference to iid resampling is clearly visible.

2.1 Data

```r
# Simulate a VAR(1) series, dropping a burn-in period
set.seed(123)
sim_var <- function(T, A, burn = 200) {
  p <- nrow(A)
  E <- matrix(rnorm((T + burn) * p), ncol = p)
  X <- matrix(0, T + burn, p)
  for (t in 2:(T + burn)) X[t, ] <- A %*% X[t - 1, ] + E[t, ]
  X[-(1:burn), ]
}

A <- matrix(c(0.8, 0.3, 0, 0.5), 2)
Xs <- sim_var(1000, A)

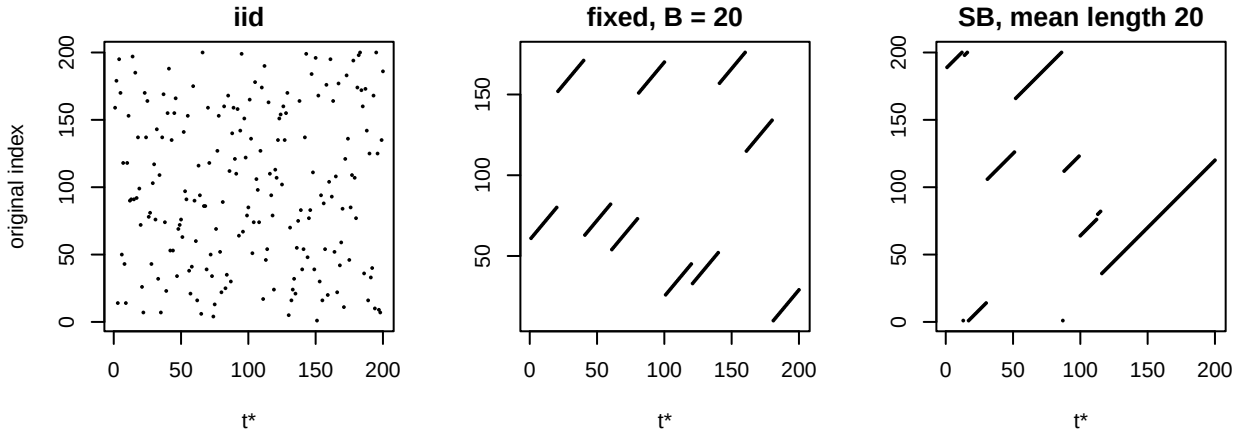
# ACF averaged over the components
avg_acf <- function(X, K = 30) {
  a <- acf(X, lag.max = K, plot = FALSE)$acf
  p <- ncol(X)
  sapply(0:K, function(k) mean(sapply(1:p, function(j) a[k + 1, j, j])))
}

# Average of avg_acf over the bootstrap samples
boot_avg_acf <- function(Zs, K = 30) {
  rowMeans(sapply(Zs, function(Z) avg_acf(Z, K)))
}
```

2.2 Index patterns

One realisation of the index vector is shown for each method with $T = 200$ and block length (or expected block length) 20.

```r
par(mfrow = c(1, 3), mar = c(4, 4, 2, 1))
set.seed(123)
plot(iid_index(200), pch = 16, cex = 0.4, main = "iid",
     xlab = "t*", ylab = "original index")
plot(fixed_block_index(200, 20), pch = 16, cex = 0.4, main = "fixed, B = 20",
     xlab = "t*", ylab = "")
plot(stationary_index(200, 1 / 20), pch = 16, cex = 0.4, main = "SB, mean length 20",
     xlab = "t*", ylab = "")
```



The iid panel shows scattered points with no structure. The fixed-block panel shows straight segments of constant slope. The SB panel shows segments of varying length, because the block lengths are geometric.

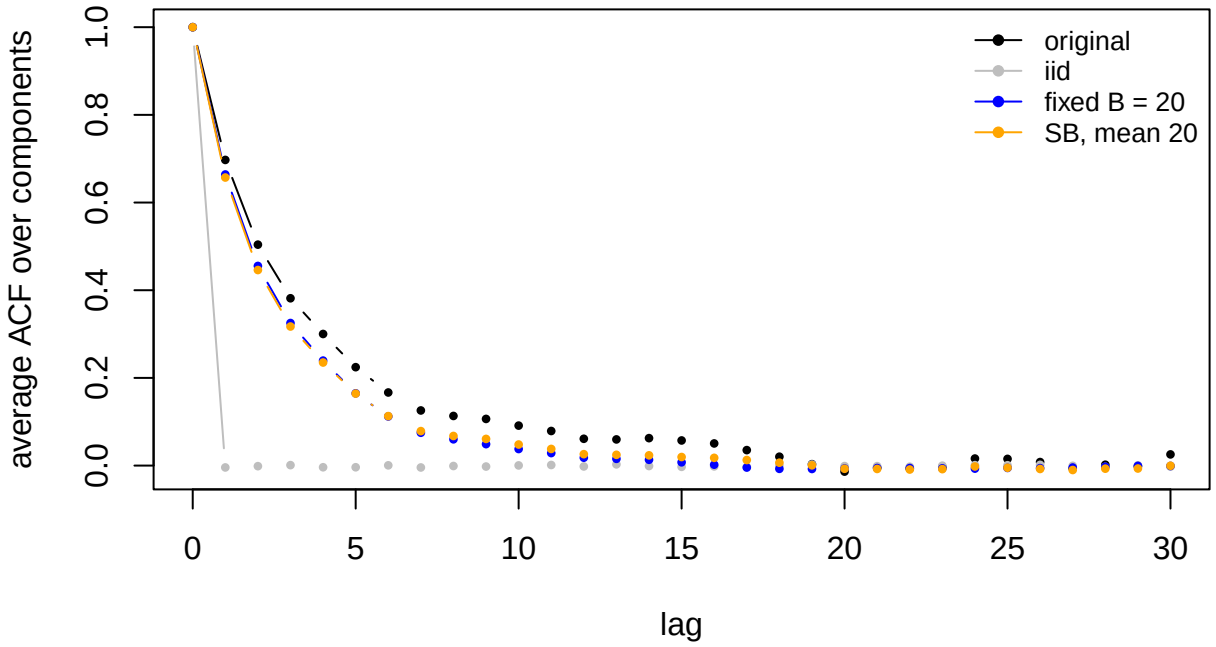
2.3 Autocorrelation of the resampled series

$M = 200$ bootstrap samples are drawn with $B = 20$ and $\ell = 20$ ($p_{\mathrm{sb}} = 0.05$).

```r
M <- 200
K <- 30
ell <- 20

a_orig <- avg_acf(Xs, K)
a_iid <- boot_avg_acf(boot_samples(Xs, M, "iid"), K)
a_fixed <- boot_avg_acf(boot_samples(Xs, M, "fixed", ell), K)
a_sb <- boot_avg_acf(boot_samples(Xs, M, "stationary", 1 / ell), K)

cols <- c("black", "gray", "blue", "orange")
par(mfrow = c(1, 1), mar = c(4, 4, 1, 1))
plot(0:K, a_orig, type = "b", pch = 16, cex = 0.6, col = cols[1],
     ylim = range(c(a_orig, a_iid, a_fixed, a_sb)),
     xlab = "lag", ylab = "average ACF over components")
lines(0:K, a_iid, type = "b", pch = 16, cex = 0.6, col = cols[2])
lines(0:K, a_fixed, type = "b", pch = 16, cex = 0.6, col = cols[3])
lines(0:K, a_sb, type = "b", pch = 16, cex = 0.6, col = cols[4])
legend("topright",
       c("original", "iid", "fixed B = 20", "SB, mean 20"),
       col = cols, lty = 1, pch = 16, cex = 0.8, bty = "n")
```



The original series has strong serial dependence (lag-1 average ACF about 0.7). The iid bootstrap reduces this to essentially zero at every non-zero lag. Both block methods are close to each other and lie slightly below the original ACF at the shorter lags (up to about lag 18), because the joins between consecutive blocks are usually not adjacent in the original series, so the local dependence is lost at those points. At larger lags all curves are close to zero.

2.4 Effect of the block length

For each B and each $\ell = B$, $M = 200$ bootstrap samples are drawn. Two summaries are reported: the average lag-1 autocorrelation over components, and the bootstrap standard deviation of the sample mean (averaged over components). The value $\ell = 1$ corresponds to the iid bootstrap.

```
# Lag-1 ACF and SD of the sample mean over M bootstrap samples
grid_row <- function(X, method, tuning, M = 200) {
  Tn <- nrow(X); pp <- ncol(X)
  ids <- replicate(M, boot_index(Tn, method, tuning), simplify = FALSE)
  Zs <- lapply(ids, function(i) X[i, ])

  acf1_avg <- mean(sapply(Zs, function(Z) {
    a <- acf(Z, lag.max = 1, plot = FALSE)$acf
    mean(sapply(1:pp, function(j) a[2, j, j]))
  }))

  means_mat <- sapply(Zs, colMeans)
  c(acf1 = acf1_avg,
    sd_mean = mean(apply(means_mat, 1, sd)))
}

ells <- c(1, 5, 10, 20, 50, 100, 200)
fx <- t(sapply(ells, function(b) grid_row(Xs, "fixed", b)))
sb <- t(sapply(ells, function(b) grid_row(Xs, "stationary", 1 / b)))

tab <- data.frame(
```

```
  ell = ells,
  acf1_fixed = fx[, 1], acf1_SB = sb[, 1],
  sd_fixed = fx[, 2], sd_SB = sb[, 2]
)
round(tab, 3)
```

```
##   ell acf1_fixed acf1_SB sd_fixed sd_SB
## 1   1     -0.004  -0.003    0.049 0.043
## 2   5      0.555   0.551    0.079 0.090
## 3  10      0.624   0.622    0.101 0.094
## 4  20      0.659   0.661    0.116 0.110
## 5  50      0.683   0.679    0.132 0.105
## 6 100      0.691   0.685    0.101 0.098
## 7 200      0.698   0.690    0.089 0.076
```

```
cat("original lag-1 average ACF:", round(a_orig[2], 3))
```

```
## original lag-1 average ACF: 0.697
```

At $\ell = 1$ both methods reduce to iid resampling, giving an ACF of about -0.004 and an SD of the mean of about 0.05. As ℓ increases, the lag-1 ACF rises from the iid value towards the original value of 0.697, reaching about 0.69 at $\ell = 200$.

The bootstrap SD of the mean peaks at moderate block lengths: around $\ell = 50$ for the fixed-block method (0.132) and $\ell = 20$ for the stationary bootstrap (0.110). For larger ℓ it decreases, dropping to about 0.08 at $\ell = 200$, where only a few blocks are drawn ($T/\ell = 5$). The two methods give very similar ACF values at every ℓ.

---

3. General check on the course data

The functions are applied to the course data `ts_data` (read in Part A) to check that they work for a general $T \times p$ matrix with $p > 2$.

```
# A few bootstrap samples of the course data
Bd <- boot_samples(ts_data, 5, "stationary", 1 / 20)
c(n_samples = length(Bd), dim = dim(Bd[[1]]))
```

```
## n_samples      dim1      dim2
##         5      1000        10
```

```
# Mean max deviation of the correlation matrix over N bootstrap replicates
set.seed(123)
N_corr <- 100
corr_dev <- function(method, tuning = NULL, N = N_corr) {
  mean(sapply(seq_len(N), function(n) {
    Z <- ts_data[boot_index(nrow(ts_data), method, tuning), , drop = FALSE]
    max(abs(cor(Z) - cor(ts_data)))
  }))
}

round(c(fixed      = corr_dev("fixed", 20),
        stationary = corr_dev("stationary", 1 / 20)), 3)
```

```
##      fixed stationary
##      0.076      0.075
```

The average maximum deviation of the contemporaneous correlation matrix is 0.077 (fixed-block) and 0.075 (stationary), so the bootstrap samples keep the correlation of the course data.

---

4. Summary of Part B

- Ordinary iid resampling destroys the serial dependence and is not appropriate for time series. Block-based resampling preserves the local dependence by resampling consecutive runs of observations.
- The fixed-block bootstrap resamples blocks of length B. The SB uses geometric block lengths with mean $\ell = 1/p_{\text{sb}}$ and wraps around, which makes it conditionally stationary. Both reduce to iid resampling when $B = 1$ or $p_{\text{sb}} = 1$.
- On a strongly dependent VAR(1) series, the block methods preserved the average ACF (slightly below the original values), while iid resampling did not. The bootstrap standard deviation of the mean was largest at moderate block lengths.
- The functions run on the 1000×10 course data.